

Cascade Integrity Checks — Instincts, Theory, and Implementation

Date: 2026-08-08 **Scope:** Two safety cross-checks for the quaternion attitude cascade (outer angle-P → inner rate-PD), plus one tuning rule to keep the outer integrator subordinate. The checks caught a real defect: a legacy-to-unified-float migration bug (see §5) that left Roll/Pitch/YawAngleQIntLimit = 10 (i.e. 10 rad/s) in nearly every MR airframe instead of the intended ~0.0026 rad/s.

1. The two safety checks (and the tuning rule)

All three are statements about *consistency between the two loops* of the cascade:

1. **Integral-limit $\leq$ max commanded rate.** $A[\text{axis}].P.\text{IntLim}$ must not exceed the axis's max commanded rate, because that rate clamp is the outer loop's only "actuator" authority. If the integral can demand more rate than the clamp will ever allow, the feedback path breaks on every saturation event and the integrator keeps charging — textbook **windup**.
 2. **P-path demand stays inside max commanded rate.** The maximum rate the outer $2 \cdot Q_a \cdot K_p$ path can ever command (at the largest attitude error) must not exceed the axis max rate. The outer loop is a pure-P mapping error → rate: if it saturates the inner loop's rate clamp before reaching its setpoint, the controller stops being a proportional tracker and degenerates toward bang-bang/limit-cycle with loss of disturbance-damping.
 3. **Tuning rule:** keep IntLim around **20-30 % of the axis max commanded rate** so the QP/P term remains the dominant rate driver and the I term only trims residual offset.
-

2. Theory — check #1 (I-limit $\leq$ rate clamp): cascade integral windup

In a two-loop cascade the inner loop is the *actuator* of the outer loop, and its saturation is the outer loop's only saturation. When the inner loop rate setpoint machines its clamp:

- The outer-loop committed-rate clamp breaks, so the integrator keeps charging toward IntLim with none of that energy being delivered → stored energy is never drained, causing overshoot and slow recovery;
- This is precisely the **cascade-type integral windup** treated in the anti-windup literature (Aström & Hägglund, 2006; Kothare et al., 1994). Standard remedies bound the integrator within the actual capability of the downstream limit — the same "conditional integration" / clamping rule PX4 applies (its rate controller keeps $_lim_int$ and clamps the integrator inside the mixer saturation; $\text{MC_RR_} * \text{INT}$ limits are sized a fraction of the torque authority).

Hence the first check: $\text{IntLim} \leq \text{axis_max_commanded_rate}$. Sizing it down to a fraction (≈ 0.2 – $0.3 \times \text{max-rate}$) further keeps the integral from steering the loop during fast setpoint changes, where the P term should dominate.

References: - PX4 multicopter rate control — integrator limit & clamping conditional integration: PX4-Autopilot `mc_rate_control` (`MC_RR_INT_LIM`), `RateControl::updateIntegral` (`i_factor` reduces I as error grows). - Kothare, Morari, et al. (1994), "An anti-windup scheme for control of saturated linear systems". Aström & Hägglund, *Advanced PID Control* (2006), Ch. 6 (windup). - Cascade wind-up literature (e.g. "Development of an Antiwindup Technique for a Cascade Control...", PMC7758956) — integral of the *primary* controller builds when the *secondary* saturates. Exactly the angle-loop/rate-loop pairing here.

3. Theory grounding — check #2 (P-path ≤ rate clamp): loop separation

The outer loop maps an attitude error to a rate setpoint. Its built P gain defines a bandwidth; standard cascade practice keeps the inner bandwidth $\geq 3\text{--}5\times$ **the outer** (loop-separation principle) so the inner loop appears as an ideal actuator to the outer. If $2 \cdot \sin(\theta_{\max}/2) \cdot K_p > \text{maxRate}$ at the maximum attitude error:

- The P path requests a rate clamped by maxRate — the commanded rate is the clamp during that error range → the outer loop operates in the saturated region well before its setpoint is reached → nonlinear, and can self-sustain a limit cycle (P/C) instead of pinching anywhere;
- Conversely the theoretical max rate the design can demand is $2 \cdot \sin(\theta_{\max}/2) \cdot K_p$, so requiring $2 \cdot \sin(\theta_{\max}/2) \cdot K_p \leq \text{maxCmdRate}$ guarantees the P path never exceeds what the inner loop can deliver.

Mathematical form (per axis):

```
maxRate_cmd = R.Max                (roll, pitch)
maxRate_cmd = min(R.Max, Nav.MaxHeadingRate) (yaw)
QP_demand   = 2 · sin(θmax/2) · Kp    (roll, pitch)
QP_demand   = 2 · 1.0 · Kp           (yaw, 180° heading error → |Qa|max=1)
```

```
Sanity #2: QP_demand ≤ maxRate_cmd
Sanity #1: IntLim ≤ maxRate_cmd
Tune rule: IntLim = 0.20–0.30 · maxRate_cmd
```

Numbers for the active Ecks quad (see §5):

Axis	maxRate_cmd (rad/s)	θmax (rad)	Kp	$2 \cdot \sin(\theta_{\max}/2) \cdot K_p$	IntLim (as-flown 10)
Roll	3.665 (af:107)	0.524	7	3.62	10 (should $\approx 0.02\text{--}0.03$)
Pitch	3.665 (af:108)	0.524	7	3.62	10 (should $\approx 0.02\text{--}0.03$)
Yaw	min(·, 1.047) = 1.047	180°	3	6.0	10 (should $\approx 0.03\text{--}0.06$)

Roll/pitch sit essentially **exactly on** the P-path boundary ($3.62 \approx 3.66$) — the tune is at the edge; yaw P demand $6.0 \gg 1.047$ means yaw params are nested strongly in the saturated region during large heading errors.

References: - Loop separation for two-loop autopilots: e.g. MDPI *Drone* loop-shaping attitude controller (inner bandwidth $3\text{--}5\times$ outer), MATLAB “Tuning of a Two-Loop Autopilot”. - “Attitude” multi-loop rate-ceiling concept — same demand P path bounded within inner-rate capability used across PX4/ArduPilot: inner rate clamp is the outer’s actuator.

4. The migration bug that flight proved

.af files store the legacy uint8 value the old param had **without** the scale that the old system applied (convert_old_defaults.py:41–43): Roll/PitchAngleQIntLimit = $P() \cdot \text{DEG_RAD} \cdot 0.015$,

$\text{YawAngleQIntLimit} = P() \cdot \text{DEG_RAD} \cdot 0.05$. So legacy 10 $\rightarrow$ intended **0.0026 rad/s** (roll/pitch) / **0.0087 rad/s** (yaw) $\rightarrow$ exactly matches GCS PARAM_DEFAULTS[4] = 0.002617. Instead the unified-float project stored the raw 10 $\rightarrow$ 10 rad/s — 3800 $\times$ over-budget, and a windup bomb with the int never self-clamping since $\text{IntLim} > \text{maxRate}$.

The checks in §1 catch exactly this class of defect.

5. Implementation

Implemented in the tuning procedure (parameter_window.py compute_defaults):

1. After computing the character-slider defaults, the method runs `_cascade_integrity_caveats()` and **issues a caveat** whenever a computed value would breach the cascade limits — it does **not** silently clamp or adopt the breach into the tune:
 - **P-path check:** $2 \cdot \sin(\theta_{\text{max}}/2) \cdot K_p > \text{maxCmdRate}$ (roll/pitch) or $2 \cdot K_p > \text{maxCmdRate}$ (yaw) $\rightarrow$ warned that the P gain saturates the rate cap;
 - **I-limit hard windup guard:** $\text{IntLim} > \text{maxCmdRate} \rightarrow$ warned that the integrator can never discharge;
 - **Tune-rule band:** $\text{IntLim} > 0.30 \cdot \text{maxCmdRate} \rightarrow$ warned it is above the 20–30% band (QP no longer the main driver). Caveats surface in the console, an amber status banner, the sim-button label, and a one-time dialog; values remain applied as-is so the user can then pull the slider or gains back inside the band. `_check_cascade_sanity()` re-runs the same checks on live widget edits (not just Compute).
2. The `IntLim` **curves are left unchanged** — the 20–30% band is a *ceiling, not a target*. Raising the slider toward 20–30% of max command is deliberately **not** warranted: a smaller I-limit is always safe (QP stays the main driver, I only trims). Manual entries above the band (like the legacy 10) are what the caveats catch.
3. `test_pid_sim.py` gains `check_cascade_integrity()` — the same P-path / I-limit cross-checks run against the loaded (or slider-applied) FC-native params in the full critique, flagging any airframe that breaches the caps (e.g. the `IntLim = 10` windup bomb) as a FAIL.

These checks are **conservative by construction**: they only *surface a caveat* when a computed value breaches a limit — they never raise gains, and they never silently clamp the tune. The upper bound is the axis max commanded rate (`R.Max`, or the `min(·, Nav.MaxHeadingRate)` for yaw); the *recommended* tune band is 20–30% of that, keeping the QP term the dominant rate driver.